

CLASES CON HERENCIA E INTERFACES

```
public interface Enviable {  
    void generarRutaEnvio(String direccion);  
}
```

```
public abstract class Producto {  
    private String nombre;          //Privado: Encapsulado  
    protected double precioBase;    //Protegido: Para los hijos  
  
    public Producto(String nombre, double precioBase) {  
        this.nombre = nombre;  
        this.precioBase = precioBase;  
    }  
  
    public String getNombre() {  
        return this.nombre;  
    }  
  
    public abstract double calcularPrecioFinal();  
}
```

```
public class Electrodomestico extends Producto implements Enviable {  
    private int mesesGarantia;  
  
    public Electrodomestico(String nombre, double precioBase, int mesesGarantia) {  
        super(nombre, precioBase); // Inicializa el padre  
        this.mesesGarantia = mesesGarantia;  
    }  
  
    @Override  
    public double calcularPrecioFinal() {  
        return this.precioBase * 1.21; // Aplica IVA  
    }  
  
    @Override  
    public void generarRutaEnvio(String direccion) {  
        System.out.println("Logística: Despachando " + getNombre() + " hacia: " +  
direccion);  
    }  
  
    public int getMesesGarantia() {  
        return this.mesesGarantia;  
    }  
}
```

PROGRAMA DE PRUEBA QUE RECORRE TODO EL ARRAY

```

public class GestionTiendaApp {
    public static void main(String[] args) {

        // Estructura Estática (Array de tamaño fijo: 3)
        Electrodomestico[] escaparate = new Electrodomestico[3];

        escaparate[0] = new Electrodomestico("Televisor 4K Smart", 400.0, 36);
        escaparate[1] = new Electrodomestico("Horno Microondas", 90.0, 24);
        escaparate[2] = new Electrodomestico("Aspiradora Robot", 250.0, 12);

        System.out.println("=== REPORTES DEL INVENTARIO ESTÁTICO ===");
        System.out.println("Capacidad fija del inventario: " + escaparate.length + "
posiciones.\n");

        for (int i = 0; i < escaparate.length; i++) {
            System.out.println("- [Posición " + i + "] Análisis de Artículo:");
            System.out.println("- Artículo: " + escaparate[i].getNombre());
            System.out.println("- Garantía del fabricante: " + escaparate[i].getMesesGarantia()
+ " meses.");
            System.out.println("- Coste de producción: " + escaparate[i].precioBase + "€");
            System.out.println("- PVP al público (con IVA): " +
escaparate[i].calcularPrecioFinal() + "€");

            escaparate[i].generarRutaEnvio("Sección de Despachos, Casillero " + i);
            System.out.println("-----");
        }
    }
}

```

```

public class GestionTiendaApp {
    public static void main(String[] args) {

        // 1. Creamos y llenamos el array igual que antes
        Electrodomestico[] escaparate = new Electrodomestico[3];
        escaparate[0] = new Electrodomestico("Televisor 4K Smart", 400.0, 36);
        escaparate[1] = new Electrodomestico("Horno Microondas", 90.0, 24);
        escaparate[2] = new Electrodomestico("Aspiradora Robot", 250.0, 12);
        System.out.println("=== MOSTRANDO UN ELEMENTO ESPECÍFICO ===");
        // 2. Acceso directo a la posición 1 (el Horno Microondas)
        int posicionDeseada = 1;

        System.out.println("Artículo seleccionado: " +
escaparate[posicionDeseada].getNombre());
        System.out.println("Precio final con IVA: " +
escaparate[posicionDeseada].calcularPrecioFinal() + "€");

        // También puedes llamar a sus métodos de interfaz directamente
        escaparate[posicionDeseada].generarRutaEnvio("Calle Falsa 123");
    }
}

```

TENIENDO DOS CLASES DISTINTAS

```

public class Periferico extends Producto {
    private String tipoConexion; // Ej: "USB", "Bluetooth"

    public Periferico(String nombre, double precioBase, String tipoConexion) {
        super(nombre, precioBase); // Inicializa el nombre y precio en el padre
        this.tipoConexion = tipoConexion;
    }

    // Cumple el contrato obligatorio de Producto
    @Override
    public double calcularPrecioFinal() {
        return this.precioBase * 1.10; // Los periféricos tienen un impuesto menor, por
ejemplo
    }

    public String getTipoConexion() {
        return this.tipoConexion;
    }
}

```

PROGRAMA DE PRUEBA

```

import java.util.Arrays;

public class GestionTiendaApp {
    public static void main(String[] args) {

        // =====
        // 1. CREAR EL ARRAY MIXTO (Polimorfismo)
        // =====
        // El array es de tipo 'Producto' (el padre), así que acepta CUALQUIER hijo
        Producto[] inventario = new Producto[4];

        inventario[0] = new Electrodomestico("Televisor 4K", 400.0, 36);
        inventario[1] = new Periferico("Ratón Gamer RGB", 25.0, "USB");
        inventario[2] = new Electrodomestico("Microondas", 90.0, 24);
        inventario[3] = new Periferico("Teclado Mecánico", 70.0, "Bluetooth");

        // =====
        // 2. BUSCAR UNA POSICIÓN CONCRETA
        // =====
        System.out.println("=== BUSQUEDA DIRECTA ===");
        int posicionDeseada = 1; // Queremos ver el Ratón
        System.out.println("En la posición " + posicionDeseada + " hay un: " +
inventario[posicionDeseada].getNombre());
        System.out.println("Precio: " + inventario[posicionDeseada].calcularPrecioFinal() +
"€\n");

        // =====
        // 3. OPCIONES PARA RECORRER Y VER EL ARRAY
    }
}

```

```
// =====

// OPCIÓN A: El bucle FOR clásico (Usa índices i)
// Ideal si necesitas saber en qué número de casilla estás en cada momento.
System.out.println("=== OPCIÓN A: BUCLE FOR CLÁSICO ===");
for (int i = 0; i < inventario.length; i++) {
    System.out.println("Índice [" + i + "]: " + inventario[i].getNombre());
}
System.out.println();

// OPCIÓN B: El bucle FOR-EACH (Por cada...)
// Es la forma más limpia y moderna en Java si solo quieres leer los datos sin
importar el índice.
System.out.println("=== OPCIÓN B: BUCLE FOR-EACH ===");
for (Producto prod : inventario) {
    // Java sabe automáticamente qué 'calcularPrecioFinal' usar según el objeto real
    System.out.println("- " + prod.getNombre() + " | PVP: " + prod.calcularPrecioFinal()
+ "€");
}
System.out.println();

// OPCIÓN C: Ver las "entrañas" del objeto (Identificar qué es cada cosa)
// Si necesitas saber si el producto del array es un Periférico o un Electrodoméstico,
usas 'instanceof'
System.out.println("=== OPCIÓN C: FILTRADO POR TIPO ===");
for (Producto prod : inventario) {
    if (prod instanceof Periferico) {
        // Convertimos temporalmente el Producto a Periferico para usar su método
        propio
        Periferico peri = (Periferico) prod;
        System.out.println("[PERIFÉRICO] " + peri.getNombre() + " usa conexión: " +
peri.getTipoConexion());
    } else {
        System.out.println("[ELECTRODOMÉSTICO] " + prod.getNombre());
    }
}
}
```

VER EL ARRAY AL INSTANTE

```
// Añade esto dentro de Producto.java para que Arrays.toString() funcione con tus objetos:
@Override
public String toString() {
    return this.nombre + " (" + this.precioBase + "€)";
}
```

Si añades eso, en tu programa principal podrás imprimir todo el array de golpe sin bucles haciendo simplemente esto:

```
System.out.println(Arrays.toString(inventario));
// Resultado en consola: [Televisor 4K (400.0€), Ratón Gamer RGB (25.0€), Microondas
```

```
(90.0€), Teclado Mecánico (70.0€)]
```

EJEMPLO COMPLETO

```
public interface Enviable {
    void generarRutaEnvio(String direccion);
}
```

```
public abstract class Producto {
    private String nombre;
    protected double precioBase;

    public Producto(String nombre, double precioBase) {
        this.nombre = nombre;
        this.precioBase = precioBase;
    }
    public String getNombre() {
        return this.nombre;
    }

    public abstract double calcularPrecioFinal();

    //SOBRESCRITURA EN EL PADRE: Si un hijo no define su propio toString, usará este.
    @Override
    public String toString() {
        return this.nombre + " (" + this.precioBase + "€)";
    }
}
```

```
public class Electrodomestico extends Producto implements Enviable {
    private int mesesGarantia;

    public Electrodomestico(String nombre, double precioBase, int mesesGarantia) {
        super(nombre, precioBase);
        this.mesesGarantia = mesesGarantia;
    }
    @Override
    public double calcularPrecioFinal() {
        return this.precioBase * 1.21; // IVA 21%
    }
    @Override
    public void generarRutaEnvio(String direccion) {
        System.out.println("Logística: Despachando " + getNombre() + " hacia: " +
        direccion);
    }
    public int getMesesGarantia() {
        return this.mesesGarantia;
    }
    //SOBRESCRITURA EN EL HIJO: Diseñamos un texto específico para
    Electrodomésticos
    @Override
    public String toString() {
```

```

        return "[Electro] " + getNombre() + " | PVP: " + calcularPrecioFinal() + "€ (Garantía: "
        + mesesGarantia + " meses)";
    }
}

```

```

public class Periferico extends Producto {
    private String tipoConexion;

    public Periferico(String nombre, double precioBase, String tipoConexion) {
        super(nombre, precioBase);
        this.tipoConexion = tipoConexion;
    }

    @Override
    public double calcularPrecioFinal() {
        return this.precioBase * 1.10; // Impuesto 10%
    }

    public String getTipoConexion() {
        return this.tipoConexion;
    }

    // SOBRESCRITURA EN EL HIJO: Diseñamos un texto específico para Periféricos
    @Override
    public String toString() {
        return "[Periférico] " + getNombre() + " | PVP: " + calcularPrecioFinal() + "€
        (Conexión: " + tipoConexion + ")";
    }
}

```

```

import java.util.Arrays;

public class GestionTiendaApp {
    public static void main(String[] args) {

        // Creamos el array polimórfico de tipo Producto (el padre)
        Producto[] inventario = new Producto[4];

        inventario[0] = new Electrodomestico("Televisor 4K", 400.0, 36);
        inventario[1] = new Periferico("Ratón Gamer RGB", 25.0, "USB");
        inventario[2] = new Electrodomestico("Microondas", 90.0, 24);
        inventario[3] = new Periferico("Teclado Mecánico", 70.0, "Bluetooth");

        System.out.println("=== VISUALIZACIÓN RÁPIDA DEL ARRAY COMPLETO ===");
        // El método Arrays.toString() recorre el array por dentro y llama al toString() de cada
        objeto automáticamente
        System.out.println(Arrays.toString(inventario));
        System.out.println("\n=== VISUALIZACIÓN DE UNA POSICIÓN ESPECÍFICA ===");
        // Si imprimes un solo elemento directamente, también usará su toString()
        personalizado
        System.out.println("Mirando la posición 1 directamente: " + inventario[1]);
    }
}

```

```
}  
}
```

=== VISUALIZACIÓN RÁPIDA DEL ARRAY COMPLETO ===
[[Electro] Televisor 4K | PVP: 484.0€ (Garantía: 36 meses), [Periférico] Ratón Gamer RGB | PVP: 27.5€ (Conexión: USB), [Electro] Microondas | PVP: 108.9€ (Garantía: 24 meses), [Periférico] Teclado Mecánico | PVP: 77.0€ (Conexión: Bluetooth)]

=== VISUALIZACIÓN DE UNA POSICIÓN ESPECÍFICA ===
Mirando la posición 1 directamente: [Periférico] Ratón Gamer RGB | PVP: 27.5€ (Conexión: USB)